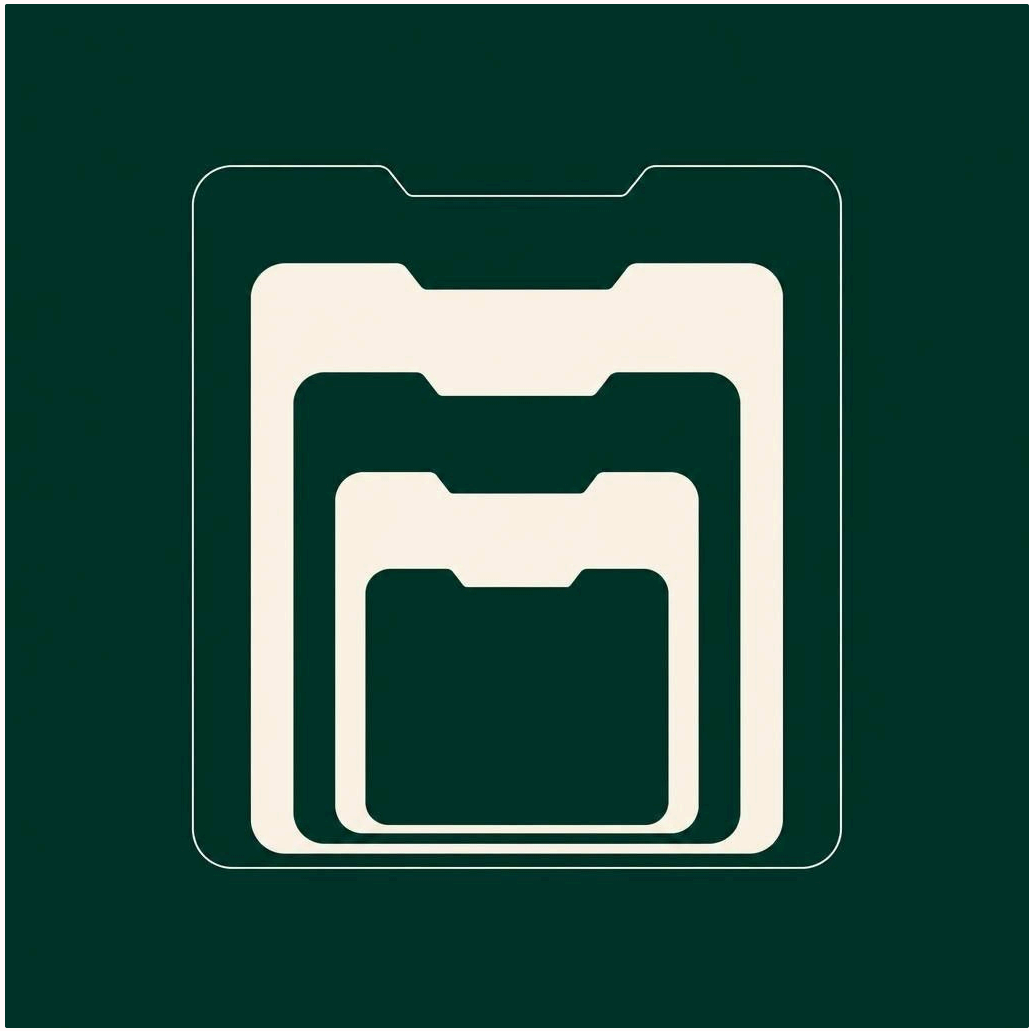
Struct Annidate in C

Struct dentro struct, funzioni e struct annidate, array e struct annidate

LINGUAGGIO C | STRUTTURE E UNIONI | STRUCT ANNIDATE



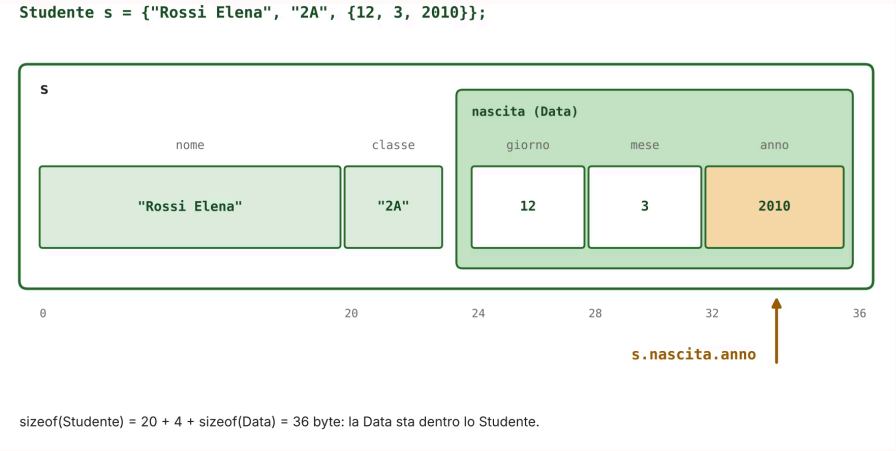
Un Campo che è una Struct

Un tipo già definito può essere il tipo di un campo: per arrivare a un campo interno i punti si leggono da sinistra a destra.

```
typedef struct {
    int giorno;
    int mese;
    int anno;
} Data;

typedef struct {
    char nome[20];
    char classe[4];
    Data nascita;
} Studente;

s.nascita.anno = 2010;
```



sizeof(Studente) = 20 + 4 + sizeof(Data) = 36 byte: la Data sta dentro lo Studente.

Data va definita prima di Studente; sizeof(Studente) comprende sizeof(Data).

Inizializzare Struct Annidate

```
Iscritto a = {"Rossi Elena", {12, 3, 2010}, {15, 9, 2025}};

Iscritto b = {.nome = "Costa Marta",
              .nascita = {.giorno = 5, .mese = 7, .anno = 2010}};

Iscritto c = {.nome = "Villa Sara", .nascita.anno = 2009};

Iscritto vuoto = {0};
```

Forma	Effetto
graffe interne	la struct interna in ordine
.nascita = {...}	la parte con il suo nome
.nascita.anno = 2009	un solo campo interno
{0}	tutto a zero, anche dentro

Le graffe interne si possono omettere, ma il compilatore avverte (-Wmissing-braces): in questo corso si scrivono sempre.

Copiare e Confrontare

```
Colloquio spostato;

spostato = originale;       /* copia anche giorno e ora */
spostato.ora = libero.ora;  /* copia un Orario intero */

/* if (spostato.giorno == altro.giorno)   errore */
stesso_momento =
    spostato.giorno.giorno == altro.giorno.giorno &&
    spostato.giorno.mese == altro.giorno.mese &&
    spostato.giorno.anno == altro.giorno.anno &&
    spostato.ora.ore == altro.ora.ore &&
    spostato.ora.minuti == altro.ora.minuti;
```

Operazione	Struct annidate
copia di tutto	b = a copia anche le parti interne
copia di una parte	b.ora = a.ora, se il tipo è lo stesso
confronto	mai con ==, a nessun livello

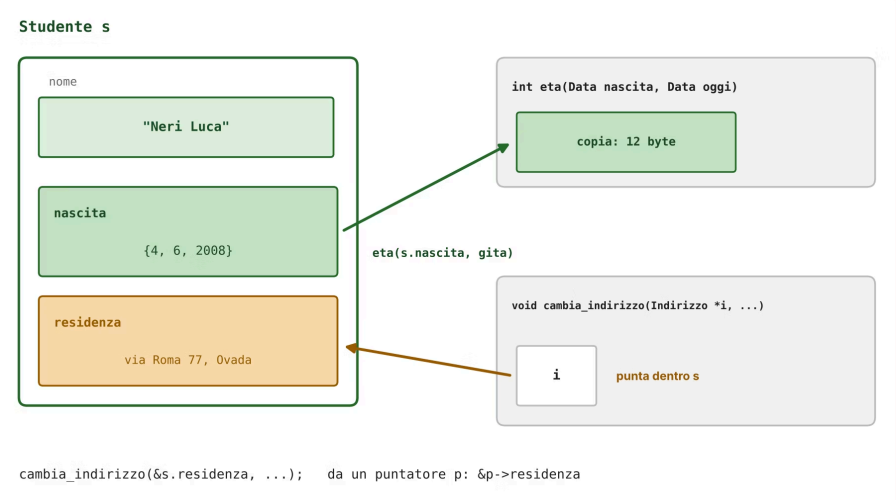
> Il confronto scende fino ai campi semplici: numeri con ==, stringhe con strcmp.

Passare una Parte alla Funzione

Una funzione che lavora sulle date riceve una Data, non uno Studente: così serve per qualunque Data, qualunque sia la struct che la contiene.

```
int eta(Data nascita, Data oggi);
void cambia_indirizzo(Indirizzo *i, const char *via,
                      const char *cap, const char *comune);

eta(s.nascita, gita);              /* copia di 12 byte */
cambia_indirizzo(&s.residenza, "via Cavour 3",
                 "15121", "Alessandria");
```



Per leggere una parte piccola basta il valore; per modificarla si passa il suo indirizzo, e la funzione non vede il resto dello studente.

Puntatori e Parti Interne

Ho in mano	Campo interno	Indirizzo della parte
Studente s	s.residenza.cap	&s.residenza
Studente *p	p->residenza.cap	&p->residenza

La freccia serve solo dove c'è un puntatore: p->residenza è già una struct, quindi dopo si usa il punto. &p->residenza si legge &(p->residenza).

```
void trasferisci(Studente *p, const char *classe,
                 const char *via, const char *cap,
                 const char *comune)
{
    strcpy(p->classe, classe);
    cambia_indirizzo(&p->residenza, via, cap, comune);
}

/* p->residenza->cap     errore */
```

Funzioni a Livelli

```
void stampa_studente(const Studente *s)
{
    printf("%s, nato/a il ", s->nome);
    stampa_data(&s->nascita);
    printf("\n  residente in ");
    stampa_indirizzo(&s->residenza);
    printf("\n");
}

int studente_valido(const Studente *s)
{
    int nome_ok = s->nome[0] != '\0';
    int data_ok = data_valida(&s->nascita);
    int indirizzo_ok = indirizzo_valido(&s->residenza);

    return nome_ok && data_ok && indirizzo_ok;
}
```

Ogni tipo ha le sue funzioni, e la funzione del tipo esterno chiama quelle dei tipi interni. Ogni funzione è corta e si occupa di un solo livello.

> Se cambia il formato delle date (12 mar 2010 invece di 12/03/2010) si modifica solo stampa_data: tutte le funzioni che la usano si adeguano.

Array di Struct in un Campo

```
typedef struct {
    char nome[4];
    Studente alunni[MAX_ALUNNI];
    int n_alunni;
} Classe;

int aggiungi(Classe *c, const char *nome, double media)
{
    int riuscito = c->n_alunni < MAX_ALUNNI;

    if (riuscito) {
        strcpy(c->alunni[c->n_alunni].nome, nome);
        c->alunni[c->n_alunni].media = media;
        c->n_alunni++;
    }
    return riuscito;
}
```

Una classe è un nome e un elenco di studenti: l'array e il suo contatore stanno insieme nella struct, e una funzione che riceve la Classe ha già tutto quello che le serve.

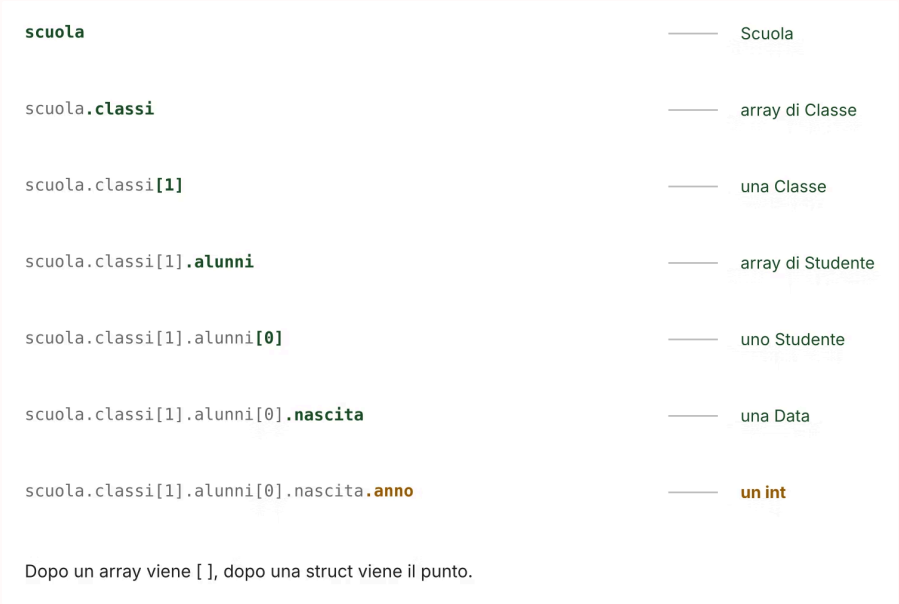
> La struct è grande (sizeof comprende tutto l'array): si passa per indirizzo, Classe * per modificarla e const Classe * per leggerla.

Leggere una Catena di Accessi

Una scuola ha un array di classi, ogni classe un array di studenti, ogni studente una data di nascita. La catena si legge un passo alla volta.

```
for (c = 0; c < scuola.n_classi; c++) {
    for (s = 0; s < scuola.classi[c].n_alunni; s++) {
        anno = scuola.classi[c].alunni[s].nascita.anno;
        ...
    }
}
```

Dopo un array viene [ ], dopo una struct viene il punto. I cicli si annidano come i livelli: classi fuori, studenti della classe c dentro, fino al contatore.



Una Funzione per Ogni Livello

La catena lunga si spezza con una funzione per ogni livello, che riceve solo il suo pezzo e chiama la funzione del livello sotto.

```
double media_classe(const Classe *c)
{
    double somma = 0;
    int s;

    for (s = 0; s < c->n_alunni; s++) {
        somma = somma + media_studente(&c->alunni[s]);
    }
    return c->n_alunni > 0 ? somma / c->n_alunni : 0;
}
```



Ogni funzione conosce un solo livello di indici.

media_studente conosce i voti, media_classe gli studenti, migliore le classi: nessuna funzione deve sapere come sono fatti i livelli più interni.

Errori Comuni da Evitare

1

Saltare un livello: s.anno invece di s.nascita.anno.

2

Usare il nome di un campo come variabile: nascita.anno.

3

Definire il tipo interno dopo il tipo che lo usa.

4

Confrontare due parti con ==, per esempio a.giorno == b.giorno.

5

Usare la freccia dopo il primo livello: p->residenza->cap.

6

Passare la parte per valore a una funzione che deve modificarla.

7

Scorrere un array interno fino a MAX invece che fino al contatore.

8

Passare per valore una struct che contiene un array grande.

Riepilogo Concetti

Situazione	Tecnica	Note
Campo che è una struct	Data nascita; dentro Studente	tipo interno definito prima
Accesso	s.nascita.anno	punti da sinistra a destra
Inizializzazione	{"Rossi", {12, 3, 2010}}	graffe interne sempre
Copia	b = a; b.ora = a.ora;	anche le parti interne
Confronto	campo per campo	mai == tra struct
Funzione su una parte	eta(s.nascita, oggi)	più generale
Modificare una parte	f(&s.residenza), f(&p->residenza)	la funzione vede solo la parte
Array dentro struct	c->alunni[i], c->n_alunni	array e contatore insieme
Più livelli	una funzione per livello	struct grandi per indirizzo

SEZIONE 1

Esempi Pratici: Struct dentro Struct

Concetti base — tre file su campi interni, inizializzazione e copia.

1 – 010-campo-struct.c

Problema

due studenti della 2A e chi è il più giovane.

Tecnica

Data dentro Studente, punti in catena e sizeof.

2 – 020-inizializzazione-annidata.c

Problema

gli iscritti al corso di robotica.

Tecnica

graffe interne, campi con nome e `{0}`.

3 – 030-copia-e-confronto-annidati.c

Problema

i colloqui con i genitori.

Tecnica

copia di tutto e di una parte, confronto campo per campo.

Esempi Pratici: Problemi su Struct dentro Struct

040 – Prestiti della biblioteca

Tre Data per prestito: giorni di ritardo e multa.

050 – Anagrafica studenti

Indirizzo dentro Studente: ricerca per comune e CAP.

060 – Pianta del laboratorio

Banchi come due Punto: area, centro e distanza dalla porta.

070 – Visite al museo

Visita con Data e Orario: controllo delle sovrapposizioni.

080 – Tratte ferroviarie

Tratta, Fermata e Orario: il collegamento che arriva prima, anche con un cambio.

SEZIONE 2

Esempi Pratici: Funzioni e Struct Annidate

Concetti base — tre file su parti, puntatori e livelli.

1 – 010-passare-una-parte.c

Problema

le autorizzazioni per la gita dei minorenni.

Tecnica

eta(s.nascita, gita) e stampa_data per ogni Data.

2 – 020-puntatore-e-campo-interno.c

Problema

uno studente cambia casa e classe.

Tecnica

p->residenza.cap, &s.residenza e &p->residenza.

3 – 030-funzioni-a-livelli.c

Problema

le schede di due nuovi iscritti.

Tecnica

stampa e controllo dello studente con le funzioni delle parti.

SEZIONE 2

Esempi Pratici: Problemi su Funzioni e Struct Annidate

040 – Rinnovo abbonamenti

aggiungi_mesi(&ab->scadenza, mesi) e avvisi di scadenza.

050 – Scheda dispositivo

Controllo separato di Hardware e Rete, indirizzi IP duplicati.

060 – Agenda verifiche

Spostare una verifica cambiando solo data e ora.

070 – Conto della gita

Importo dentro Voce: totale e quota per studente.

080 – Validazione iscrizione

Una funzione di lettura per ogni parte, con il codice della parte sbagliata.

SEZIONE 3

Esempi Pratici: Array e Struct Annidate

Concetti base — tre file su array interni, catene e livelli.

1 – 010-array-di-struct-in-un-campo.c

Problema

la classe del corso serale con 4 posti.

Tecnica

Classe con alunni[] e n_alunni, aggiungi e media.

2 – 020-catena-di-accessi.c

Problema

lo studente più giovane del corso serale.

Tecnica

scuola.classi[c].alunni[s].nascita.anno con due cicli.

3 – 030-funzioni-sul-livello-giusto.c

Problema

il premio alla classe con la media più alta.

Tecnica

media_studente, media_classe e migliore a cascata.

SEZIONE 3

Esempi Pratici: Problemi su Array e Struct Annidate

040 – Orario settimanale

Matrice di Lezione in ogni classe: ore dei docenti e conflitti.

050 – Torneo a squadre

Squadre con la rosa: gol, rose e capocannoniere.

060 – Itinerario della gita

Tappe con due Orario: controllo della sequenza e tempi.

070 – Menu della mensa

Settimana di menu da tre piatti: calorie e piatti ripetuti.

080 – Registro di classe

Voti con data e materia: medie con filtri facoltativi.